grants are extensible, so new grant types can be added when one of the existing grants does not fit.

What about the implicit grant?

The original definition of OAuth2 included a variation on the authorization code grant known as the *implicit grant*. In this grant, the AS returned an access token directly from the authorization endpoint, so that the client didn't need to call the token endpoint to exchange a code. This was allowed because when OAuth2 was standardized in 2012, CORS had not yet been finalized and so a browser-based client such as a single-page app could not make a cross-origin call to the token endpoint. In the implicit grant, the AS redirects back from the authorization endpoint to a URI controlled by the client, with the access token included in the fragment component of the URI. This introduces some security weaknesses compared to the authorization code grant, as the access token may be stolen by other scripts running in the browser or leak through the browser history and other mechanisms. Since CORS is now widely supported by browsers, there is no need to use the implicit grant any longer and the OAuth Security Best Common Practice document (https://tools.ietf.org/html/draft-ietf-oauth-security-topics) now advises against its use.

An example of obtaining an access token using the ROPC grant type is as follows, as this is the simplest grant type. The client specifies the grant type (`password` in this case), it's client ID (for a public client), and the scope it's requesting as POST parameters in the `application/x-www-form-urlencoded` format used by HTML forms. It also sends the resource owner's username and password in the same way. The AS will authenticate the RO using the supplied credentials and, if successful, will return an access token in a JSON response. The response also contains metadata about the token, such as how long it's valid for (in seconds).

```
$ curl -d 'grant_type=password&client_id=test    #A
[CA]&scope=read_messages+post_message    #A
[CA]&username=demo&password=changeit'     #B
```